

Module 5 - Understanding Processes with Data Flow Diagrams

TASK CHECKLIST

- ☒ Use requirements elicitation techniques (interview, JAD session, questionnaire, document analysis, and observation).
- ☒ Apply requirements analysis strategies as needed to discover underlying requirements.
- ☒ Develop the requirements definition.
- ☒ Develop use cases.
- ☐ Develop data flow diagrams.
- ☐ Develop entity relationship model.
- ☐ Normalize entity relationship model.

Use Case Name: Create preliminary custom drone order	ID: UC-6	Priority: High
Actor: Customer		
Description: The customer selects and customizes a commercial drone to purchase		
Trigger: Customer wants to purchase a commercial drone		
Type: ☒ External ☐ Temporal		
Preconditions: 1. The customer is authenticated by logging in to his account 2. The Sales System Order Processing application is online		
Normal Course: 1.0 Order a customized drone 1. The customer selects a base model drone from a list of models 2. The system provides availability status for that model (in stock, out of stock) 3. For out of stock status, system displays expected date available a. Customer accepts future availability date; proceed to step 4 b. Customer rejects future availability date; return to step 1 4. The system displays a list of options and upgrades for the selected model 5. The customer selects desired model options and upgrades 6. Preliminary order with cost estimate is created and displayed 7. Customer may return to step 4, confirm order, save for future consideration, or exit without saving 8. Unconfirmed orders are stored in Unconfirmed Custom Order datastore 9. Confirmed orders are saved in Confirmed Custom Order datastore 10. <u>Shop manager is notified of Confirmed Order requiring approval</u>		
Postconditions: 1. Unconfirmed order is stored in Unconfirmed Custom Order datastore 2. Confirmed order is stored in Confirmed Custom Order datastore 3. Shop manager sent notice of Confirmed Order requiring approval		

FIGURE 4-1 Create preliminary custom drone order use case—casual format.

A process explained in plain text

The **shop manager** specifies the **ID for the order** he must approve. The system retrieves the **order details**. Using the **custom components** on the order, the shop manager should “**Verify Drone Configuration**”. Information about the custom components is retrieved from **data storage** and used to perform the weight and balance check and the battery check for the custom drone. If problems are found, the shop manager can search for **alternative components** until the modified custom configuration passes those verification checks. After making changes to the drone configuration that pass the verification checks, the **modified order** is stored temporarily while the **customer is contacted for approval** of the order change. **If the customer accepts** the change, the changes are recorded in the **custom drone order** data store and the **shop manager is notified** of the customer's approval. **If the customer does not approve** the change, the **order is canceled**, and the **Customer Service Department is notified** of the cancellation so that a follow-up contact can be made.

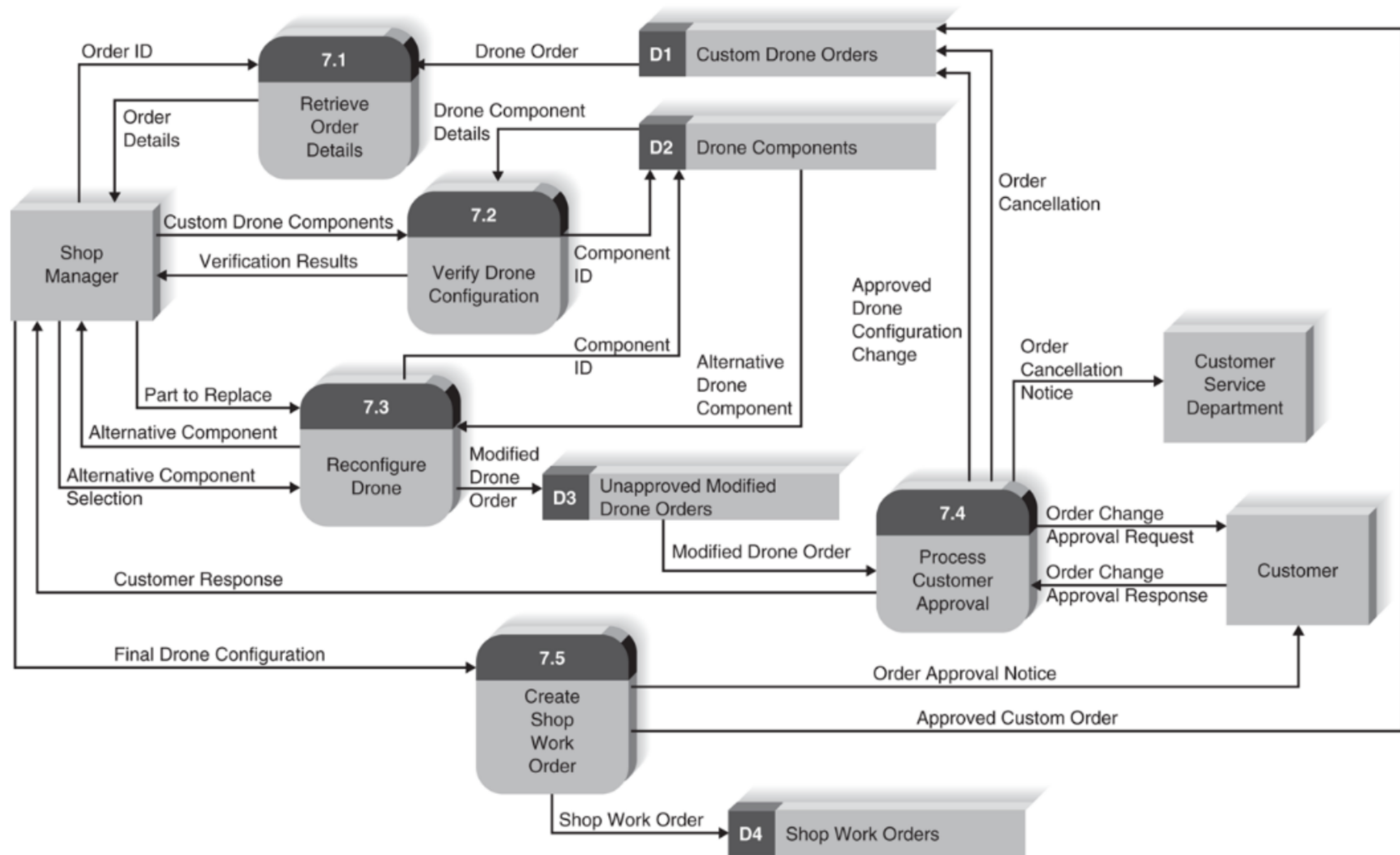


FIGURE 4-12 Shop manager approval of custom drone order level 1 DFD.

Elements of Data Flow Diagrams

Process

- An activity or function
- Can be manual or computerized
- Each process performs only one activity
- Must have at least one input data flow and at least one output data flow (otherwise how could it do something with no input and why would we consider the process that does nothing?!)

Data Flow Diagram Element	Typical Computer-Aided Software Engineering Fields	Gane and Sarson Symbol	DeMarco and Yourdon Symbol
Every <i>process</i> has a number a name (verb phase) a description at least one output data flow at least one input data flow	Label (name) Type (process) Description (what is it) Process number Process description (structured English) Notes		

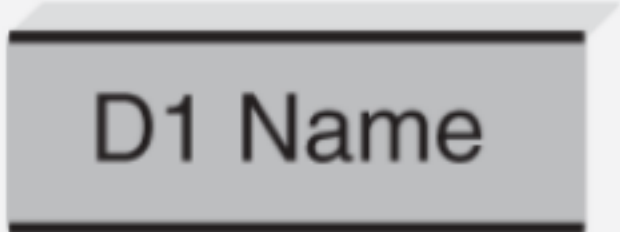
Data Flow

- A single fact or a logical collection of several facts
- The glue that holds the processes together
- One end of a data flow would always come from or go to a process. (otherwise, why would that data flow? It is either generated or consumed.)

Data Flow Diagram Element	Typical Computer-Aided Software Engineering Fields	Gane and Sarson Symbol	DeMarco and Yourdon Symbol
Every <i>data flow</i> has a name (a noun) a description one or more connections to a process	Label (name) Type (flow) Description Alias (another name) Composition (description of data elements) Notes		

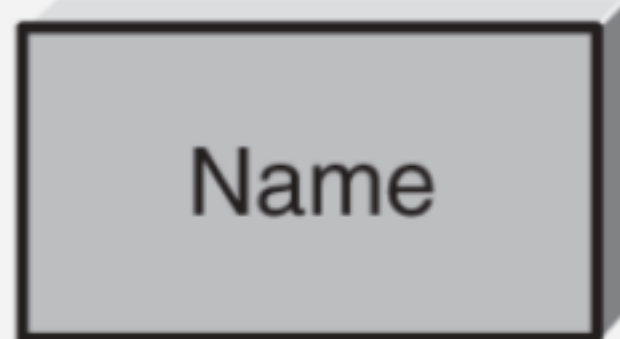
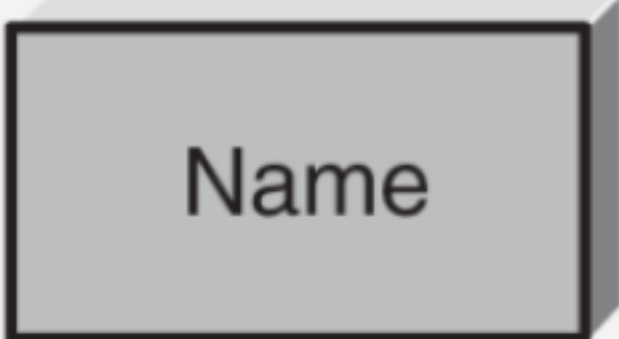
Data Store

- A collection of data
- Must contain one input data flow (or else they never contain any data)
- Must have at least one output data flow (why store data if you never use it?)

Data Flow Diagram Element	Typical Computer-Aided Software Engineering Fields	Gane and Sarson Symbol	DeMarco and Yourdon Symbol
Every <i>data store</i> has a number a name (a noun) a description one or more input data flows one or more output data flows	Label (name) Type (store) Description Alias (another name) Composition (description of data elements) Notes		

External Entity

- A **person**, **organization** unit, or **system** that is **external** to the system, but **interacts** with it.
- Typically the **primary actor** identified in a use case.
- **Provides data** to the system or **receives data** from the system
- Establishes the system **boundary**

Data Flow Diagram Element	Typical Computer-Aided Software Engineering Fields	Gane and Sarson Symbol	DeMarco and Yourdon Symbol
Every <i>external entity</i> has a name (a noun) a description	Label (name) Type (entity) Description Alias (another name) Entity description Notes		

We use data flow diagrams to
define **processes**.

A set of DFDs

- Most processes are **too complex** to be explained in one DFD, so we develop a set of DFDs:
 - First DFD → a summary of the overall system (**Context Diagram**)
 - **Additional DFDs** → more and more detail about each part of the process
- We say we **decompose** a process into a **hierarchy of DFDs**.

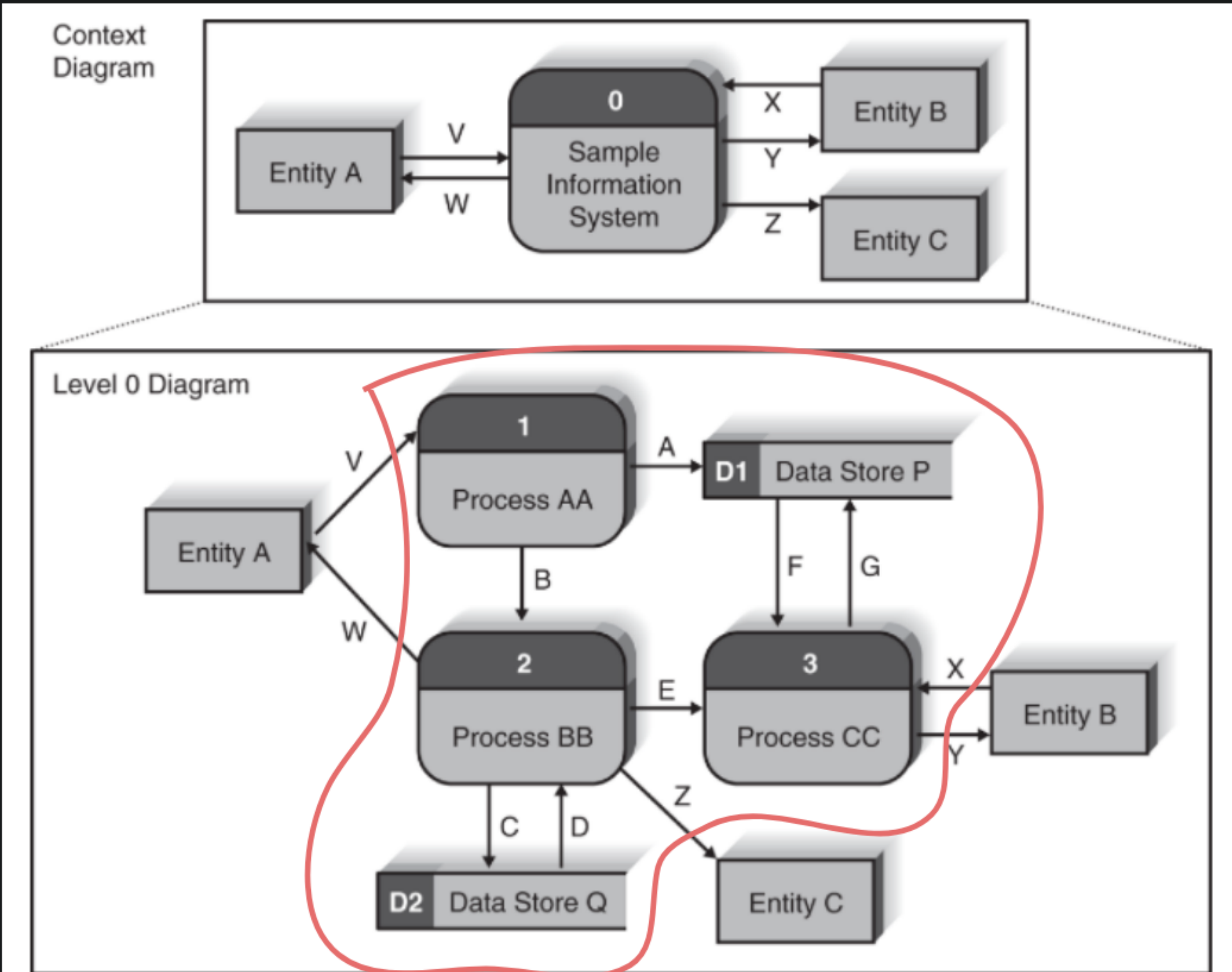
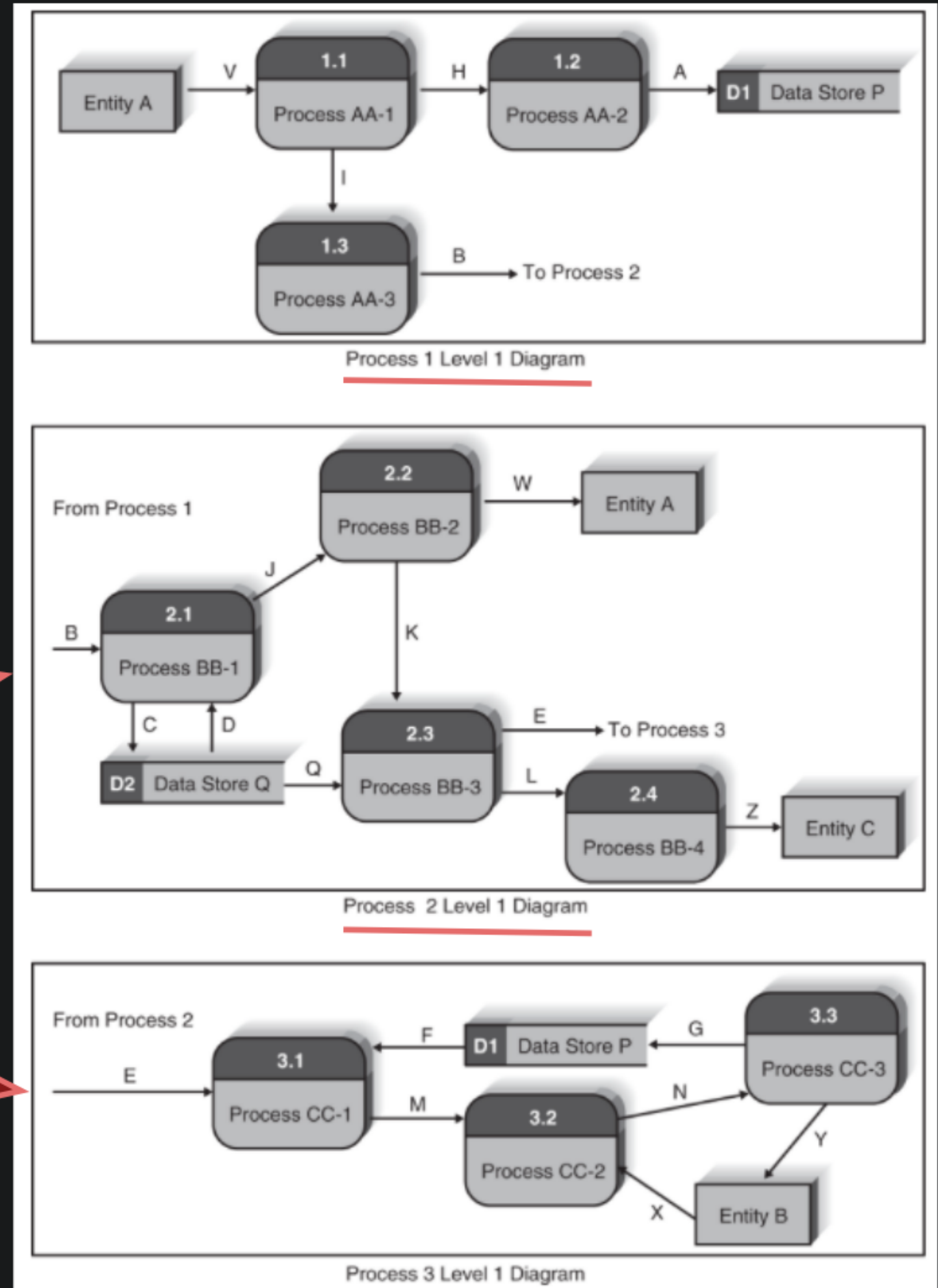
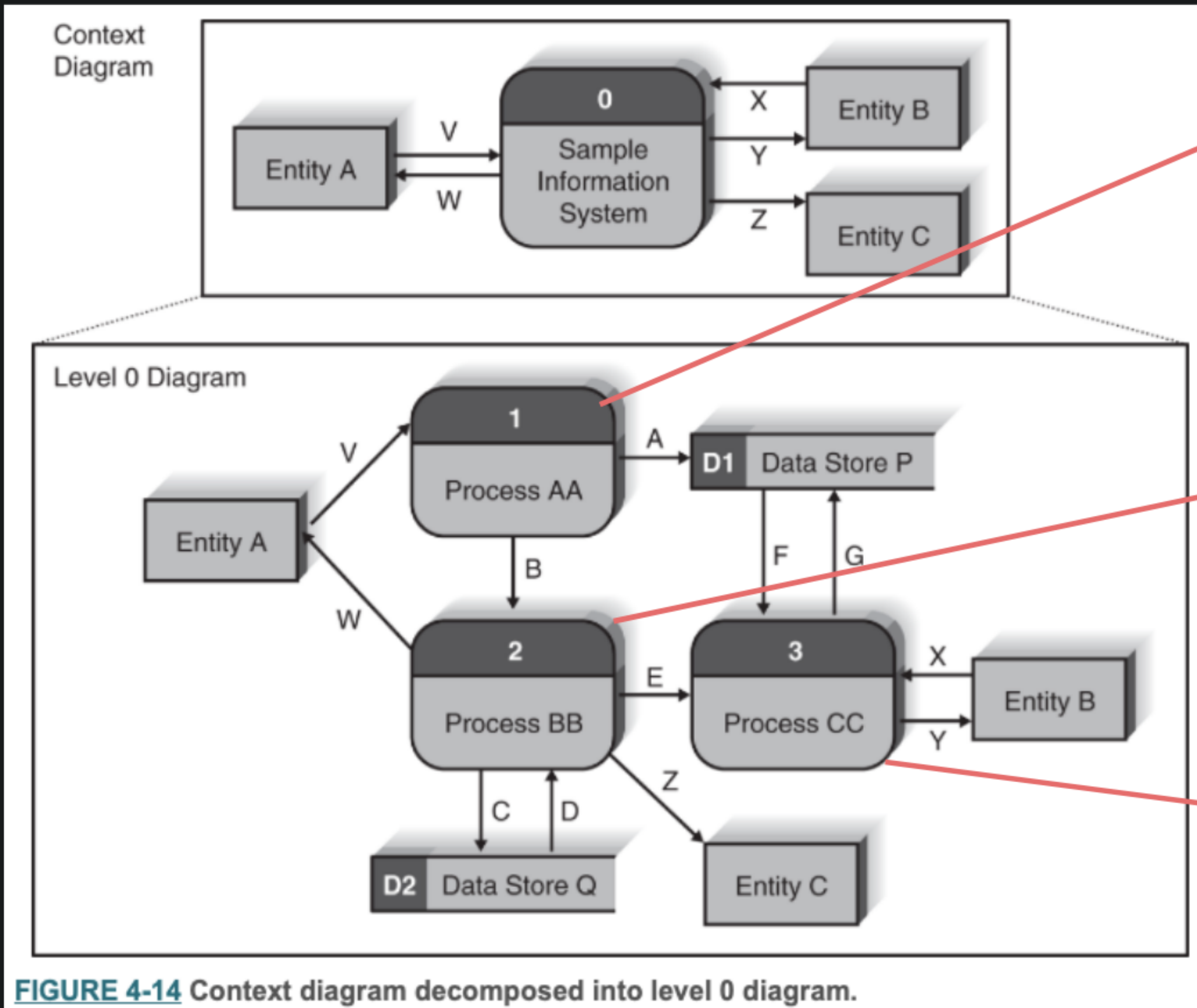
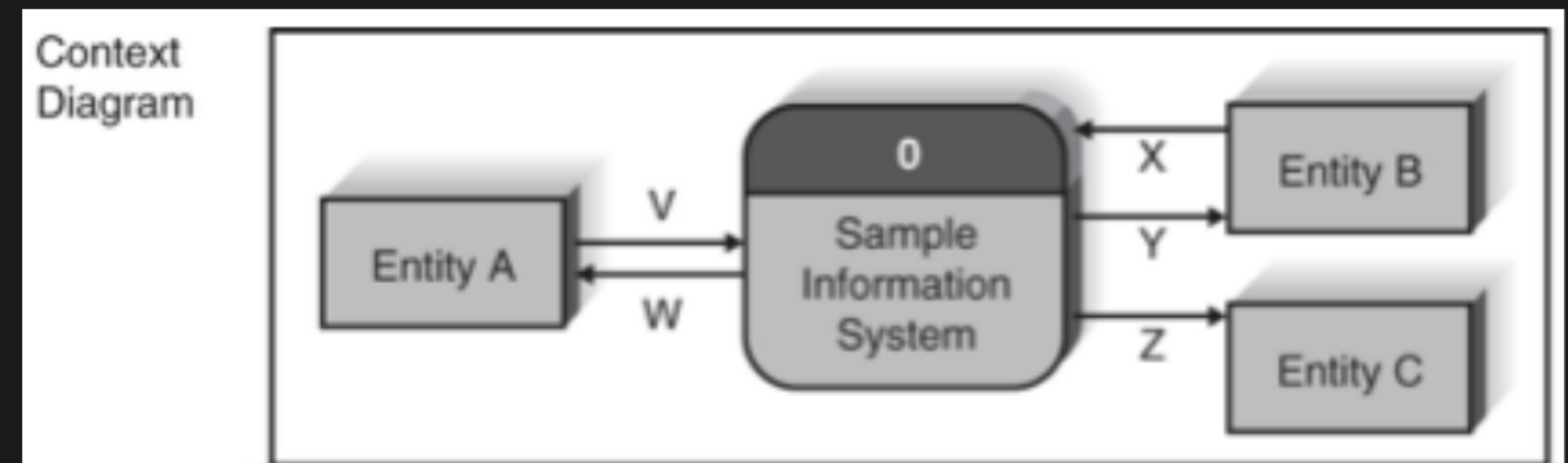


FIGURE 4-14 Context diagram decomposed into level 0 diagram.



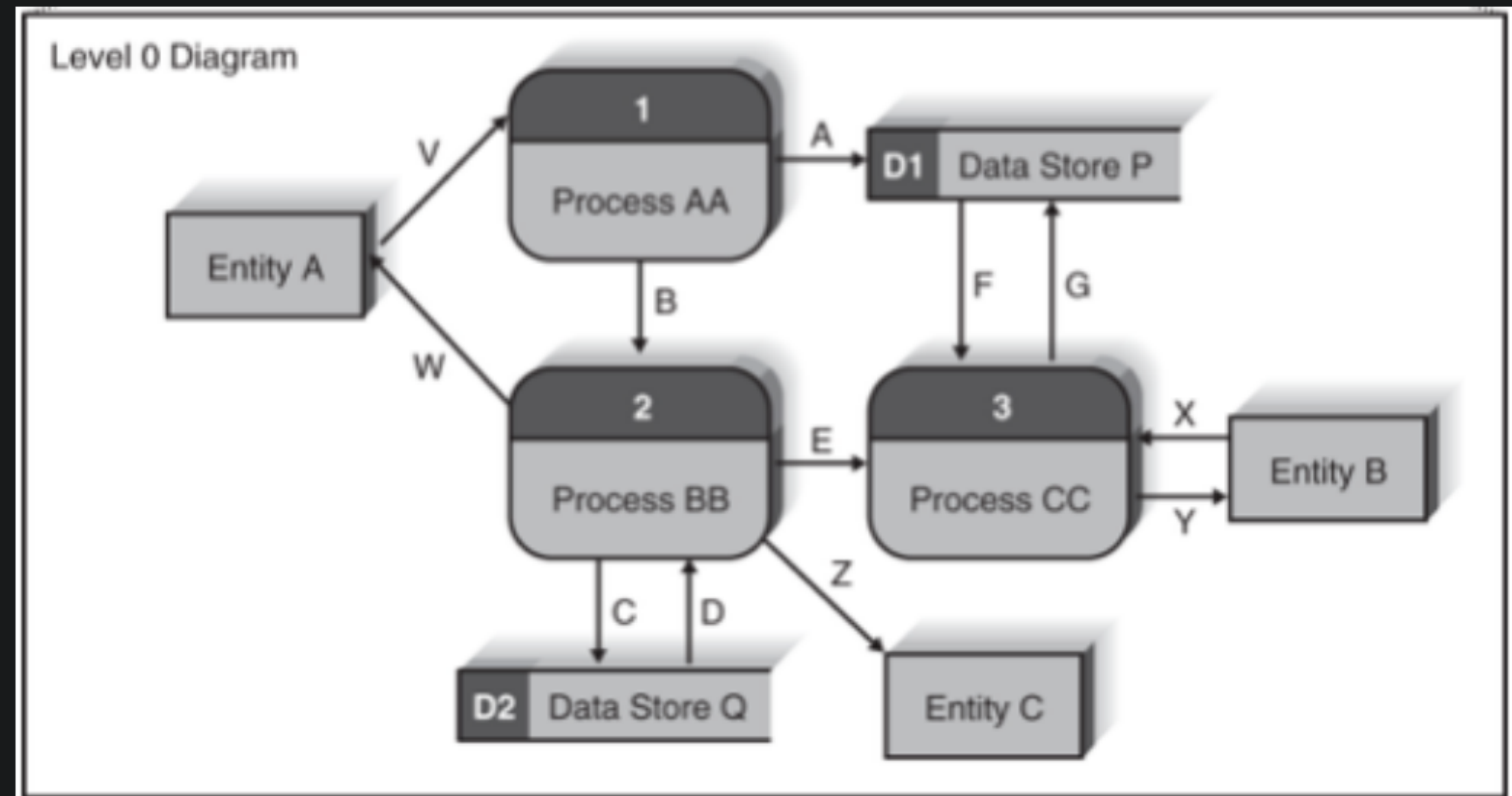
Context Diagram

- Top level DFD
- Shows the entire system **in context with its environment**
- All process models have **one context diagram** → a single process (i.e. the system itself)
- Data flows to and from external entities
- Data stores are not included.



Level 0 Diagram

- All the **major high-level processes** of the system and how they relate to each other, to external entities, and to stored data
- All process models have **one and only one level 0 DFD**.
- Level 0 diagrams and context diagrams are **balanced**: all information presented in the context diagram is accurately represented in the Level 0 DFD.
- Numbering: 1, 2, 3, etc



Level 1 Diagrams

- Each process on the level 0 DFD must be **decomposed** into a more explicit DFD, called a level 1 diagram, or level 1 DFD.
- As many level 1 diagrams as there are **major processes** on level 0 DFD
- The processes in level 1 diagram are the **children of the parent process** in the level 0 diagram.
- The set of children and the parent are **identical**.
- Ensure that the level 0 and level 1 DFDs are balanced.

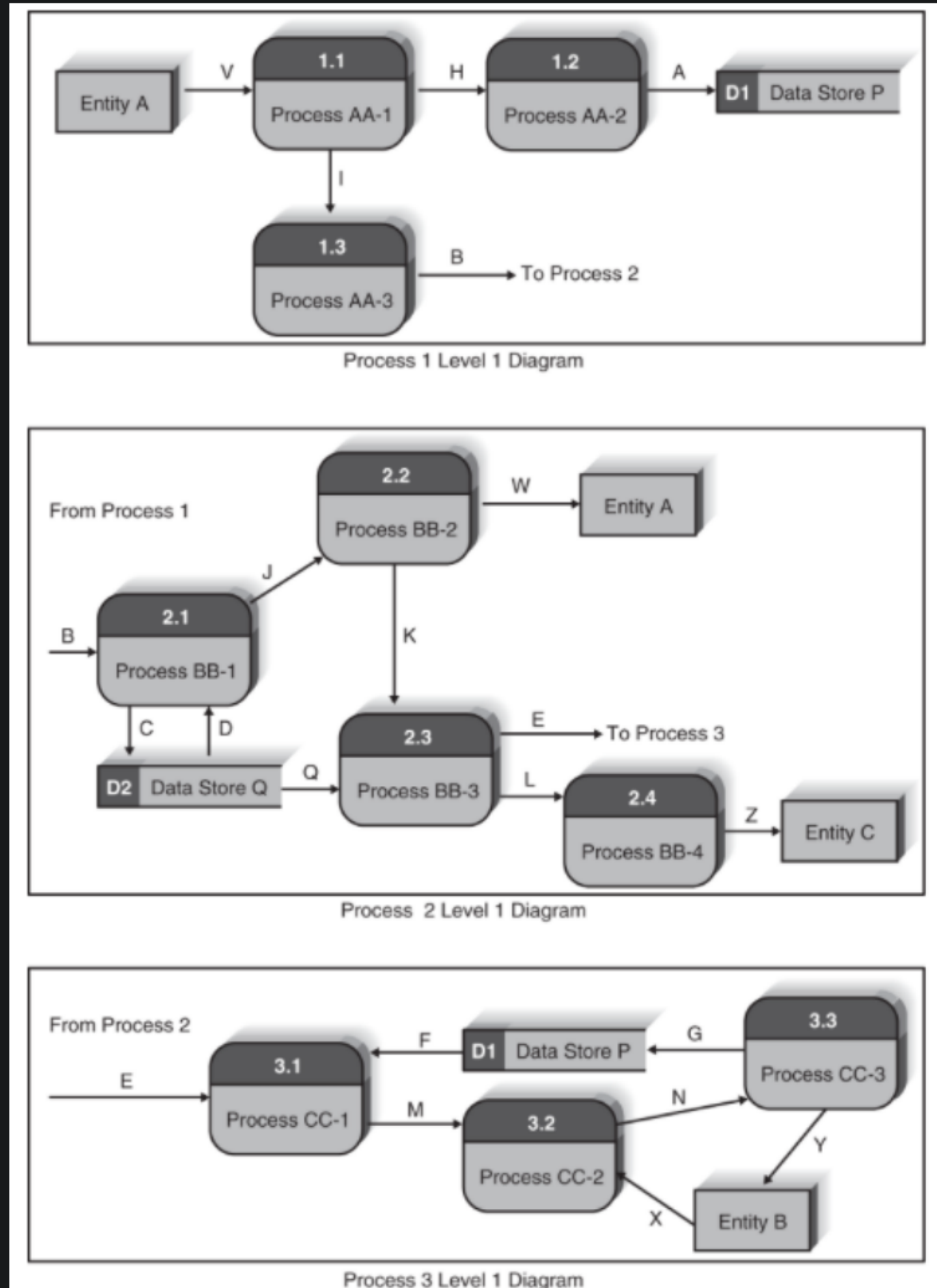


FIGURE 4-15 Processes 1, 2, and 3 level 1 diagrams.

Ultimate goal (of designing process models): to decompose a process so that each child process performs **one essential task**, not multiple tasks. Therefore we might have **level 2 diagrams and below**.

Process descriptions explain the logic that occurs inside the process when the process is complex. (e.g. when there are alternative data flows)

Creating Data Flow Diagrams

Creating Data Flow Diagrams

- Start with the information in the **use cases** and the **requirements definition**
- Take the use cases and **rewrite them as DFDs**.
- **Note:** not all processes in the requirements definition are selected as use cases. So the **process model integrates the individual use cases and adds in any remaining processes from requirements definition**.
- A useful method:
 - a. Build the **context** diagram
 - b. Create a DFD **fragment** for each use case
 - c. Organize the fragments into a **level 0** diagram
 - d. Develop **level 1 DFDs (or below)** based on the use case steps.
 - e. **Validate** the set of DFDs to make sure they are complete and correct.

Shop Management System

Three use cases (Figure 4-9):

1. Create Parts Request
2. Record Receipt of Component
3. Finalize Parts Request

Event	Response
1. New shop work order is received	• Shop work order arrival time is recorded
2. Shop work order is assigned to qualified technician	• Work order added to technician's work assignment
3. Parts Request for base model drone and additional components generated	• Parts request stored • Inventory department receives Parts Request
4. Requested parts arrive in Shop Parts room	• Parts arrival recorded on Parts Request
5. All parts on a Parts Request are available	• Technician notified that all parts are available
6. Technician records date/time work begins on work order	• Shop work order updated with start date/time
7. Technician records completion of work on work order	• Shop work order updated with completion date/time • Customer is notified of order completion

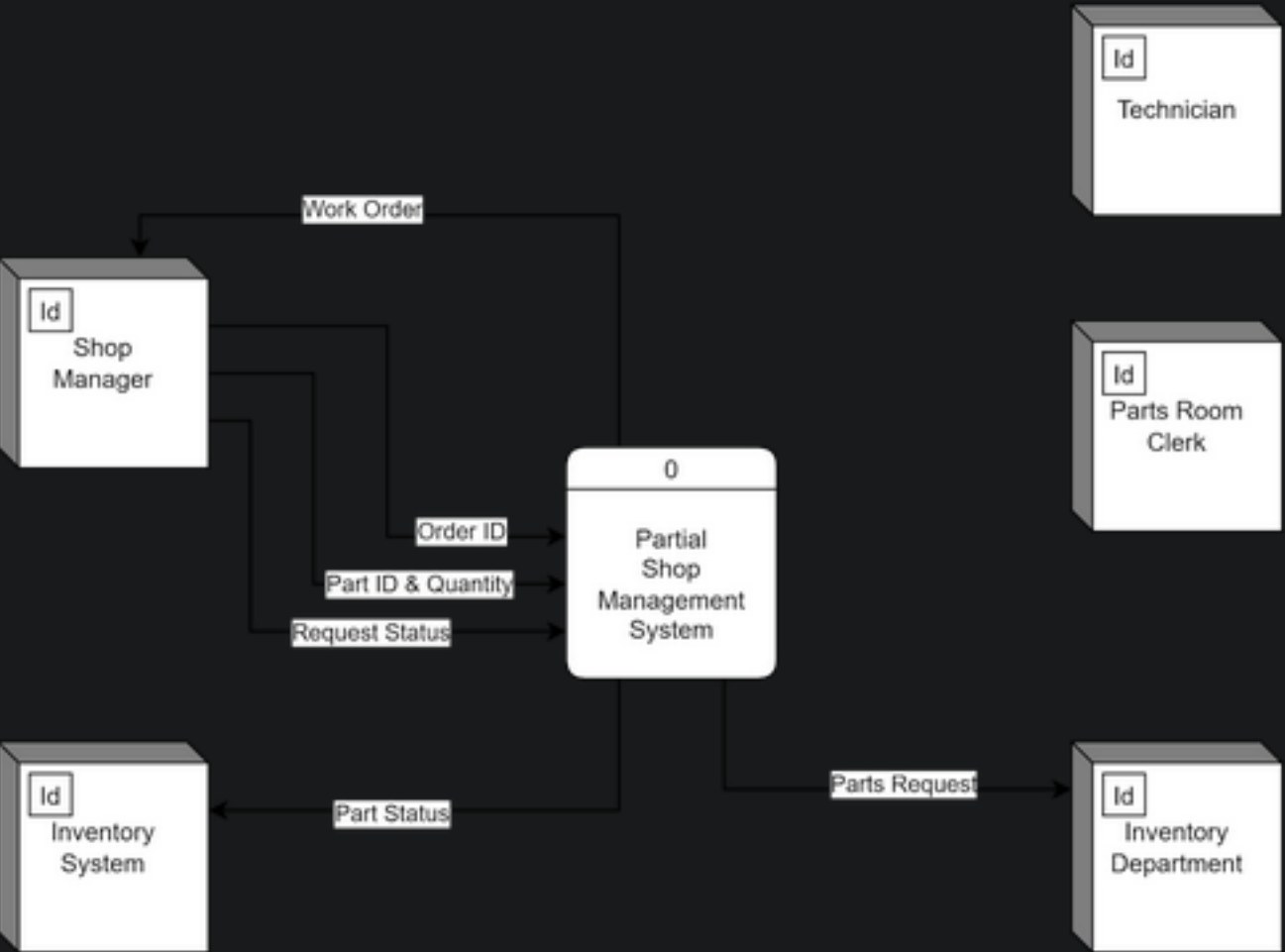
FIGURE 4-5 Sample event-response list.

Creating the context diagram

Use Case Name: Create Parts Request		ID: UC-3	Priority: High
Actor: Shop Manager			
Description: This use case describes how the Shop Manager creates a Parts Request			
Trigger: Shop Manager receives notice of new shop work order arrival from Sales System			
Type: ☒ External ☐ Temporal			
Preconditions: 1. Shop manager is authenticated 2. Parts Request application is available and on-line 3. Inventory application is available and on-line			
Normal Course: 1.0 Create Parts Request 1. Shop manager retrieves the shop work order 2. Shop manager opens a blank Parts Request 3. For each required component part a. Shop manager enters component part and quantity needed b. System queries the inventory datastore and records part status: (In stock/ Out of stock with expected date available) 4. Shop manager verifies Parts Request is complete 5. System stores new Parts Request 6. System transmits notice of Parts Request to be filled to Inventory Department		Information for Steps → Shop work order ID ← Shop work order → Part ID, quantity needed ← Part status in inventory → Part Request status → New Parts Request record → Parts Request to fill	
Postconditions: 1. New Parts Request record created and stored 2. Inventory Department notified of Parts Request to be filled			
Summary Inputs	Source	Summary Outputs	Destination
Shop work order ID Part ID and quantity Part request status	Shop manager Shop manager Shop manager	Shop work order Part inventory status New part request record Parts Request to fill	Shop manager Inventory system Parts Request datastore Inventory department

Use Case Name: Record Receipt of Component		ID: UC-4	Priority: High
Actor: Shop Parts Room Clerk			
Description: This use case describes how the Shop Parts Room clerk records delivery of component from Inventory department.			
Trigger: Drone component arrives in Parts Room from Inventory department			
Type: ☒ External ☐ Temporal			
Preconditions: 1. Parts room clerk is authenticated 2. Parts Request application is available and on-line			
Normal Course:		Information for Steps	
1.0 Record Receipt of Component		→ Part Request ID	
1. Parts Room clerk retrieves correct Parts Request		← Part Request record	
2. Parts Room clerk verifies part is correct and undamaged			
a. For incorrect part, enter "Incorrect part-returned" on Delivery slip and return to Inventory department		→ Part return to Inventory Department	
b. For damaged part, enter "Damaged part-rejected" on Delivery slip and return to Inventory department		→ Part rejection to Inventory Department	
c. Close Part Request and exit use case			
3. System records date/time part is received		→ Date/time received	
4. Parts Room clerk records the Parts Room location where the part will be stored until pickup		← Parts room storage location	
5. System stores updated Parts Request		→ Updated Parts Request	
6. System checks to see if all parts on the Parts Request are available; if they are, the Parts Room clerk is notified to perform Finalize Parts Request use case		→ Part Request Complete notification	
Postconditions: 1. Parts request updated with received part 2. Returned/rejected parts sent back to Inventory department			
Summary Inputs	Source	Summary Outputs	Destination
Part Request ID Parts room storage location	Parts room clerk Parts room clerk	Part Request record Updated Parts Request Parts Request Complete notice	Parts room clerk Parts Request datastore Parts room clerk

Use Case Name: Finalize Parts Request		ID: UC-5	Priority: High										
Actor: Shop Parts Room clerk													
Description: This use case describes how the Parts Room clerk finalizes a Parts Request													
Trigger: Notification received that all parts are available for a Parts Request													
Type: ☒ External ☐ Temporal													
Preconditions: 1. Parts room clerk is authenticated 2. All components listed on Parts Request have arrived in Parts Room													
Normal Course: 1.0 Finalize Parts Request 1. Parts room clerk opens the Parts Request and the associated Shop Work Order 2. Parts room clerk verifies that all listed components are currently in the Parts Room 3. Parts room clerk changes the Parts Request Status to "complete" 4. System records the completion date/time in the Shop Work Order 5. System notifies the assigned technician that job is ready		<table><tr><th colspan="2">Information for Steps</th></tr><tr><td>←</td><td>Parts Request record Shop Work Order record</td></tr><tr><td>→</td><td>Final parts verification</td></tr><tr><td>→</td><td>Date/time completed</td></tr><tr><td>→</td><td>Work order ready</td></tr></table>		Information for Steps		←	Parts Request record Shop Work Order record	→	Final parts verification	→	Date/time completed	→	Work order ready
Information for Steps													
←	Parts Request record Shop Work Order record												
→	Final parts verification												
→	Date/time completed												
→	Work order ready												
Postconditions: 1. Parts Request status is complete 2. Date/time all parts are available recorded in Shop Work Order 3. Technician notified that work order is ready													
Summary Inputs	Source	Summary Outputs	Destination										
Final parts verification Date/time completion	Parts room clerk Parts room clerk	Parts request record Shop work order record Work order ready notice	Parts room clerk Shop work order datastore Technician										



Creating the Context Diagram

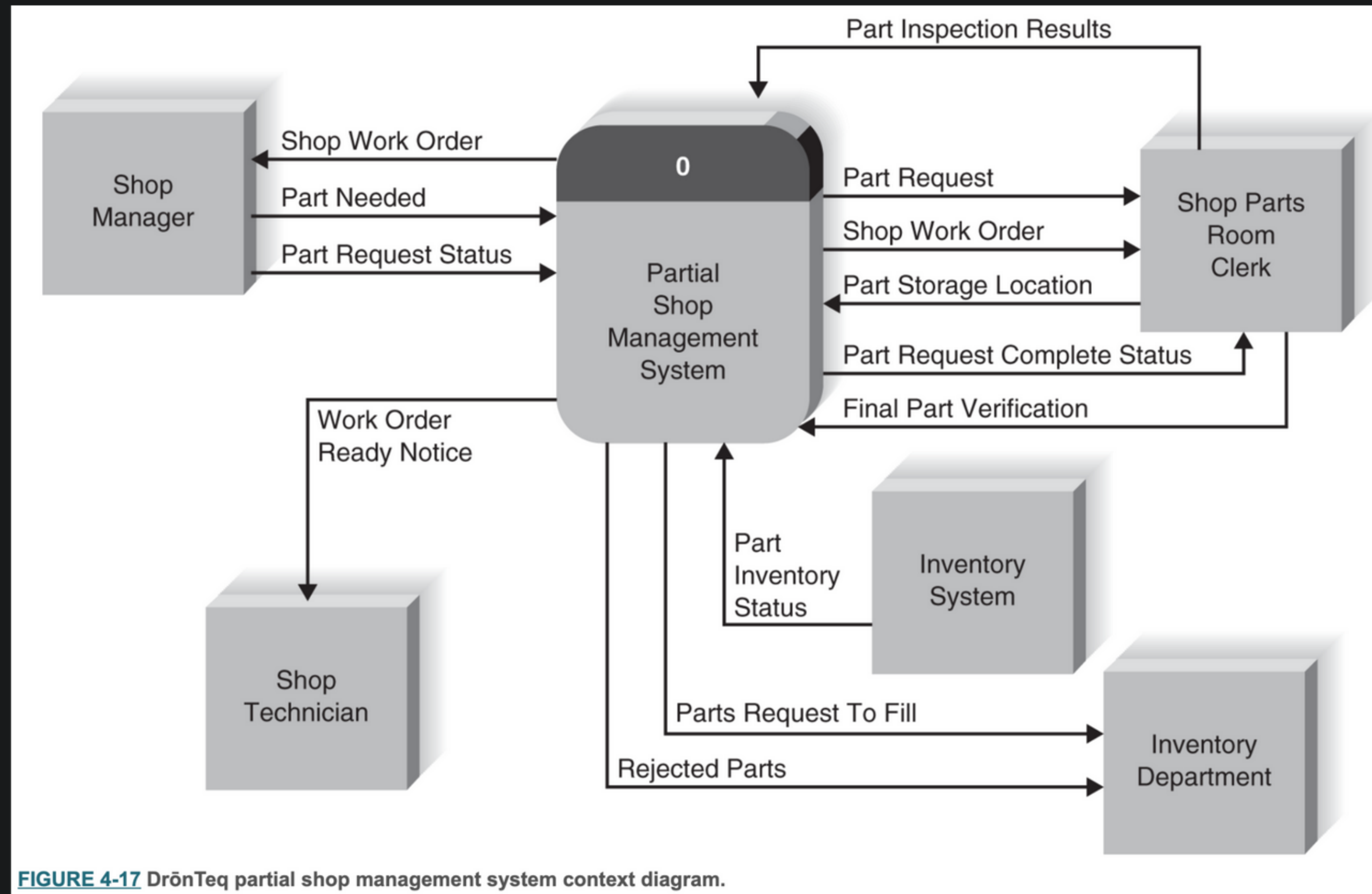
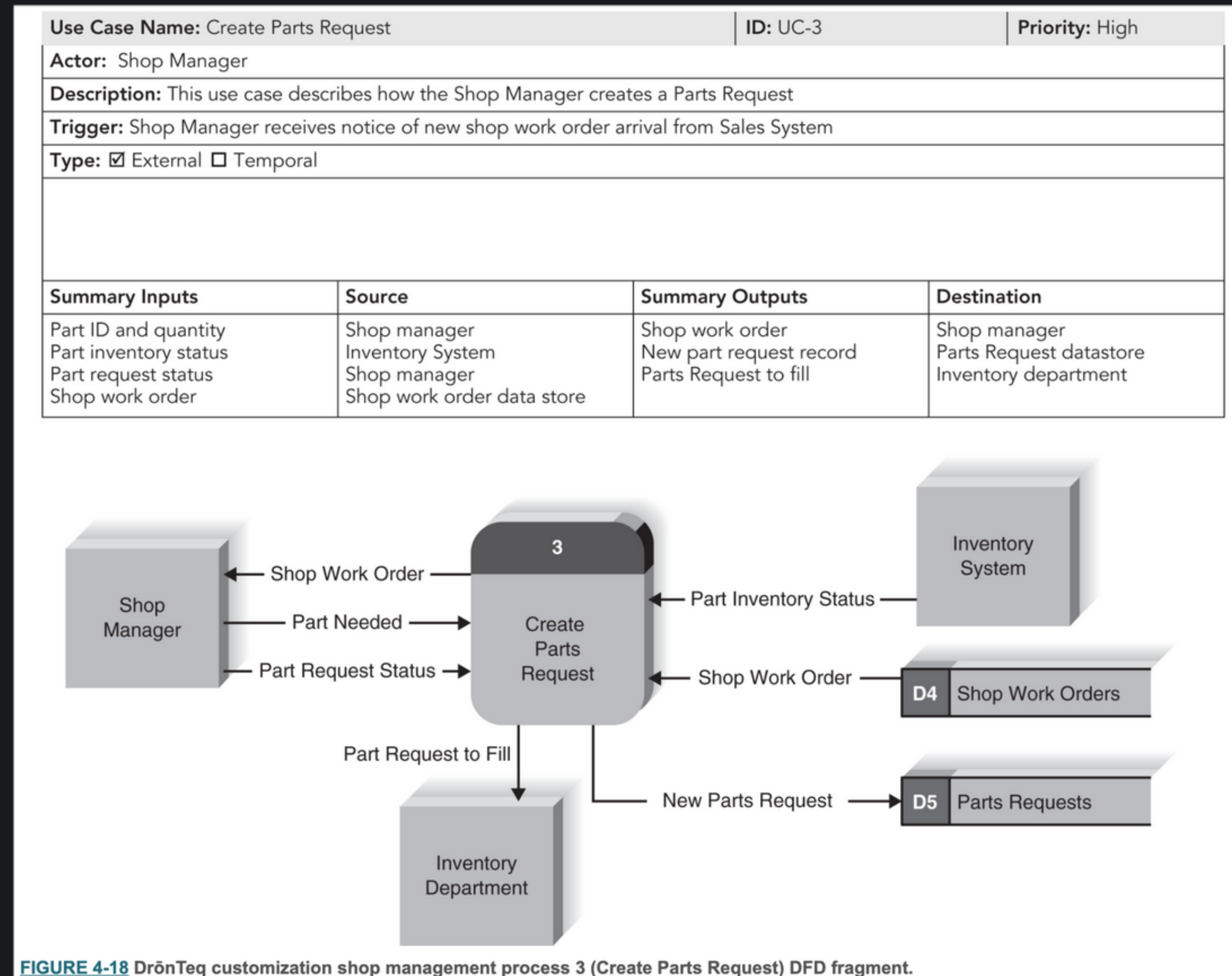


FIGURE 4-17 DrönTeq partial shop management system context diagram.

□ Your Turn: Context Diagram

Creating Data Flow Diagram Fragments

- Each use case is converted into one DFD **fragment**: one part of a DFD that eventually will be combined with other fragments to form a new DFD.
- Important to have a **consistent viewpoint** when naming processes.
- **DFD fragment vs Use Case**:
 - All process names **must be a verb phrases** on DFDs. (Use case names might not be verb phrases)
 - DFD must add the **data flows necessary** to generate the outputs. (Use cases might not list data flows read from a data store.)



Your Turn: DFD Fragments

Creating the Level 0 Data Flow Diagram

- **Combine DFD fragments** into one DFD drawing that becomes the level 0 DFD.
- Try to reduce the number of times that data flow **lines cross**. Or if they must cross, reduce confusion by using **line jumps**:

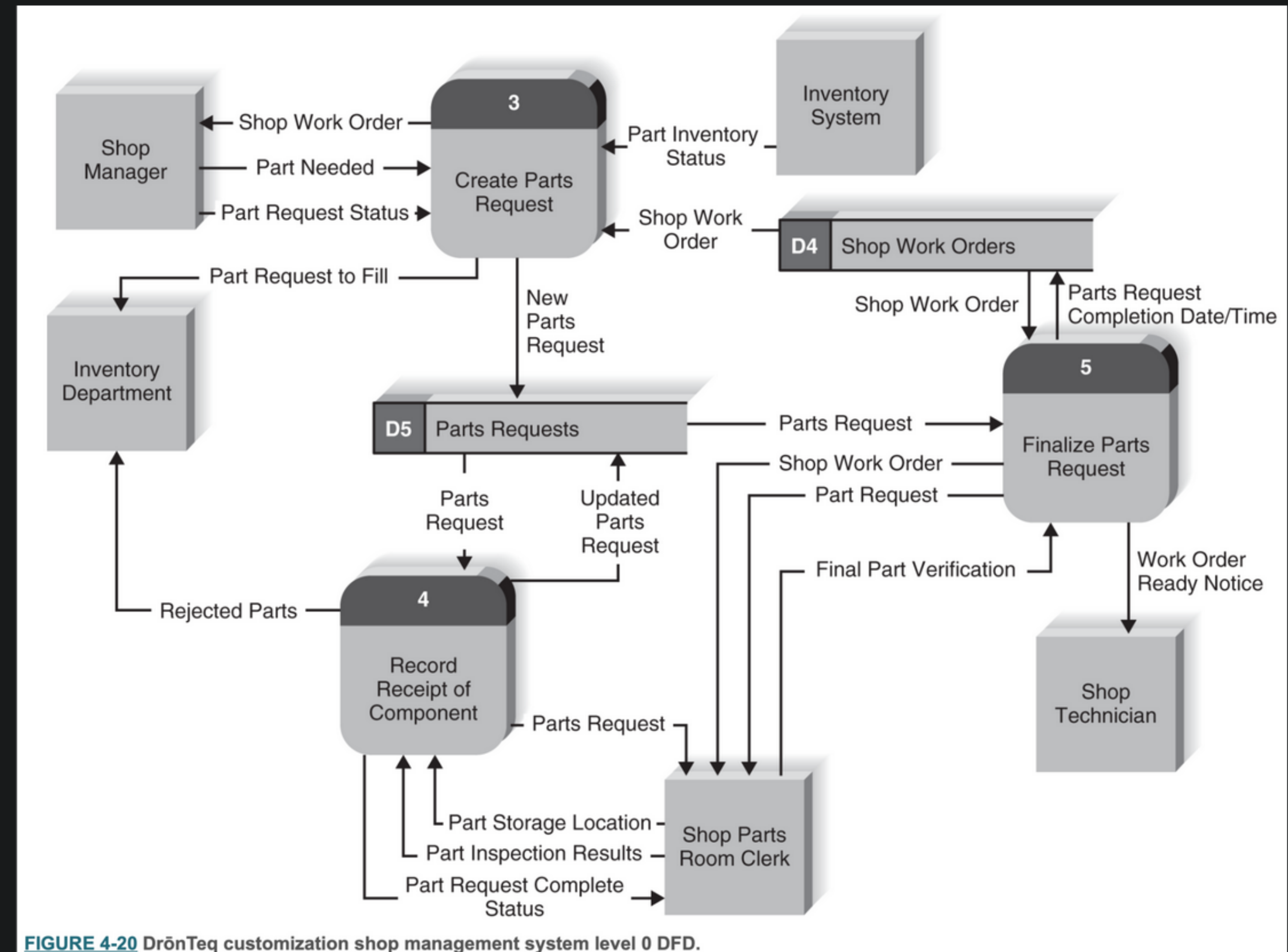
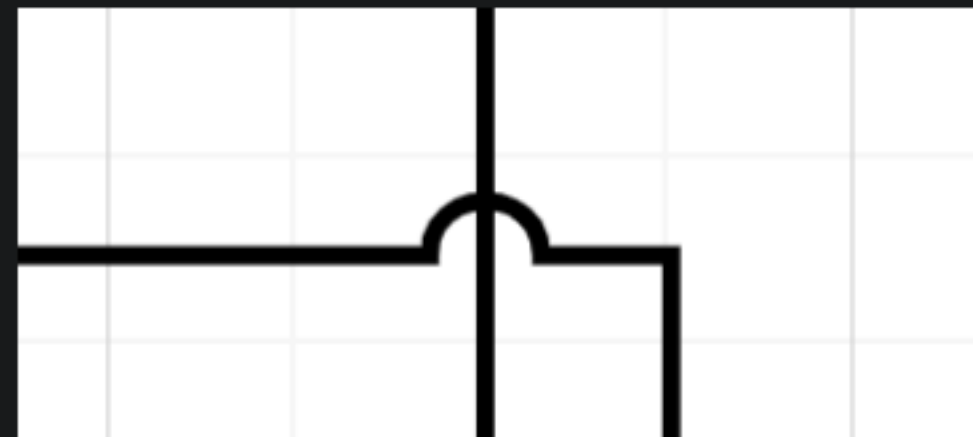
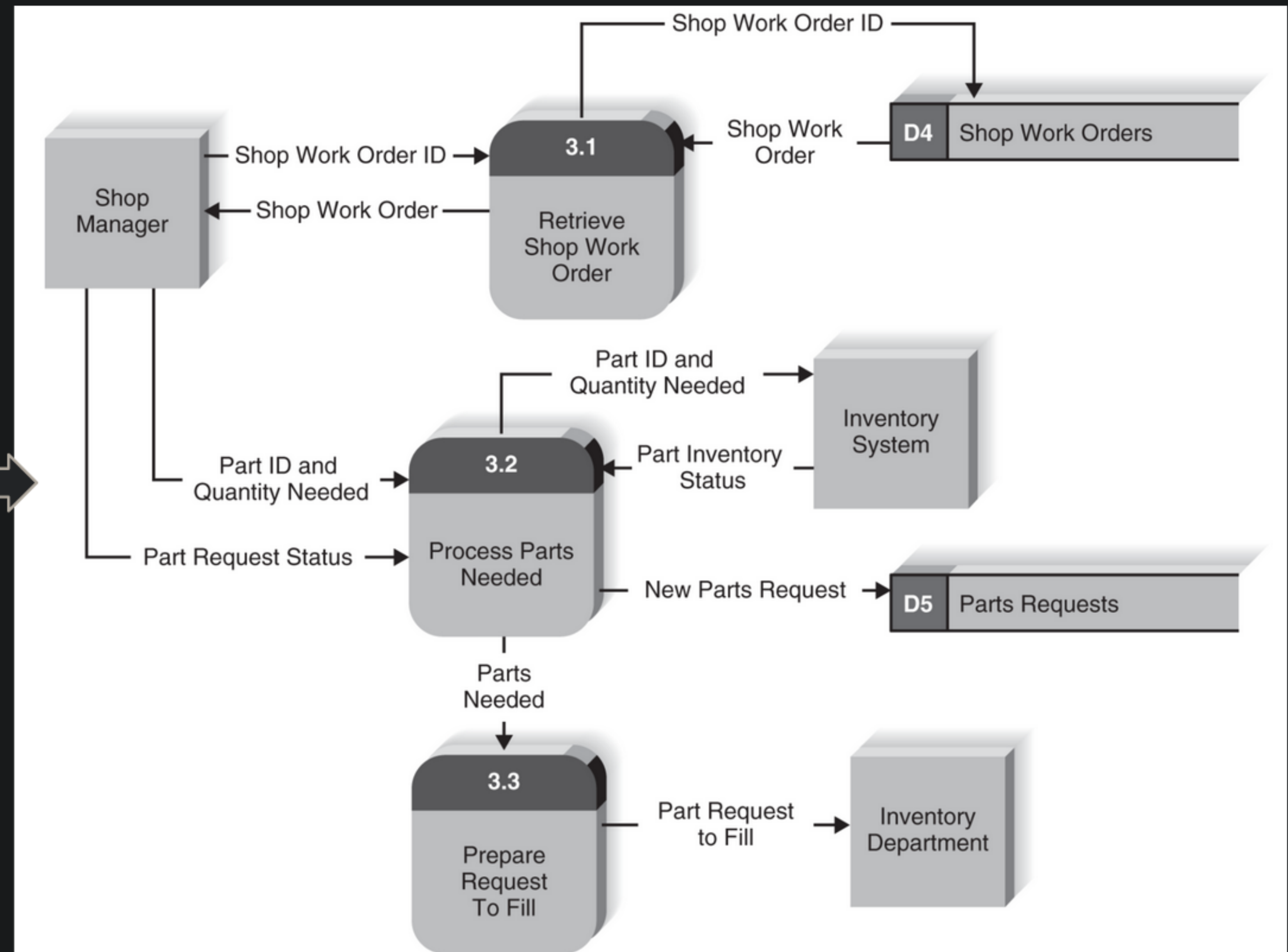
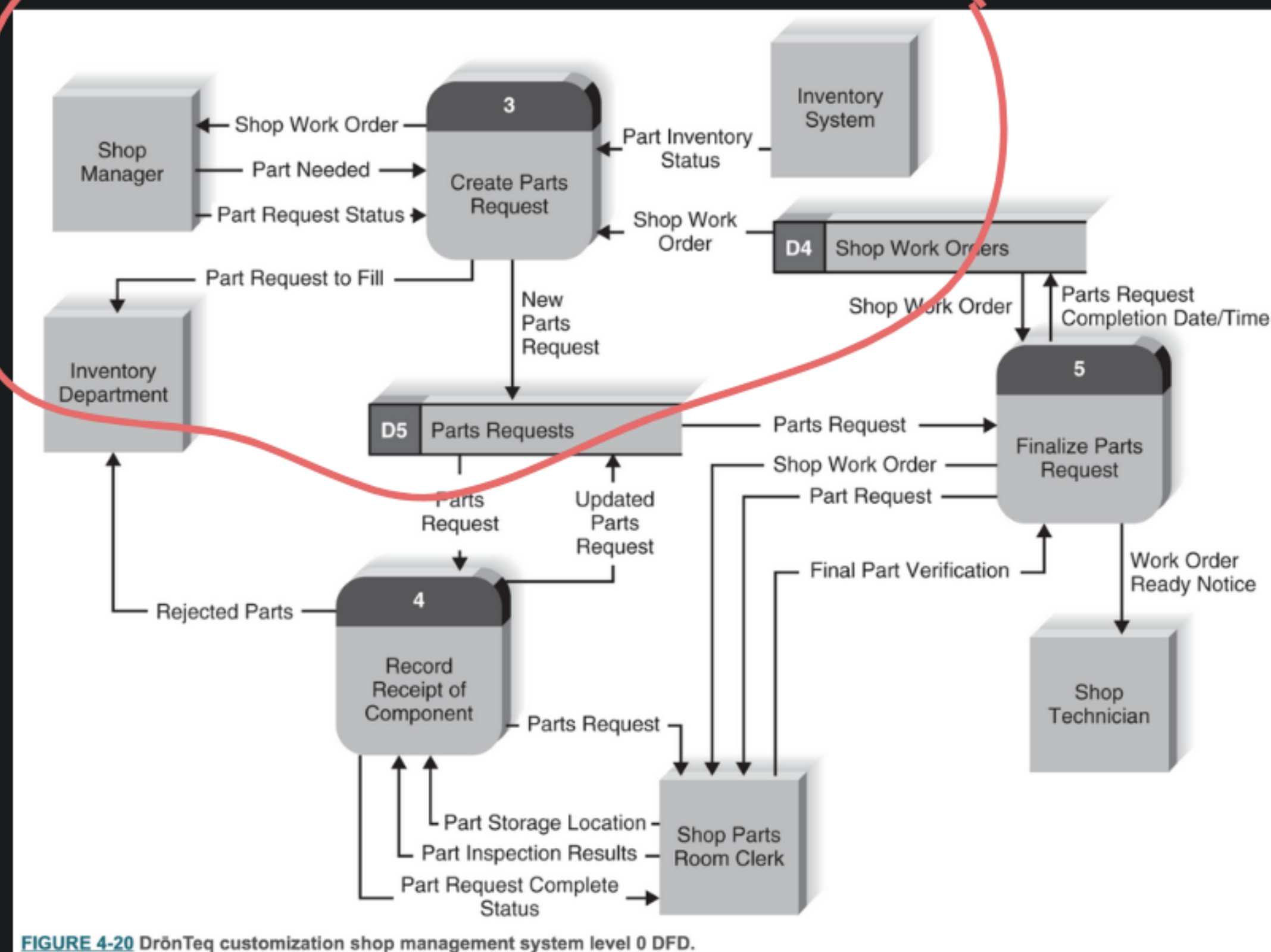
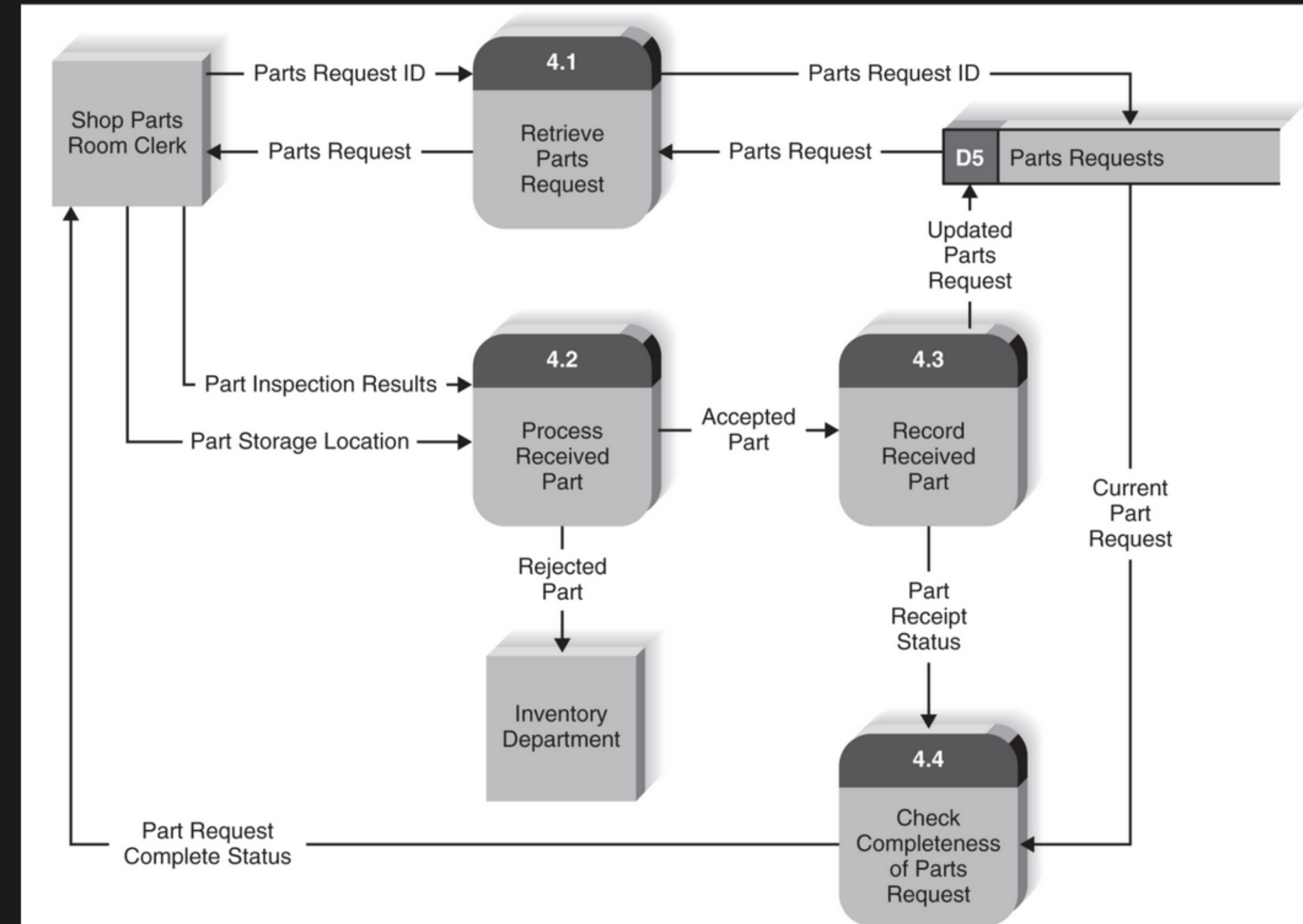
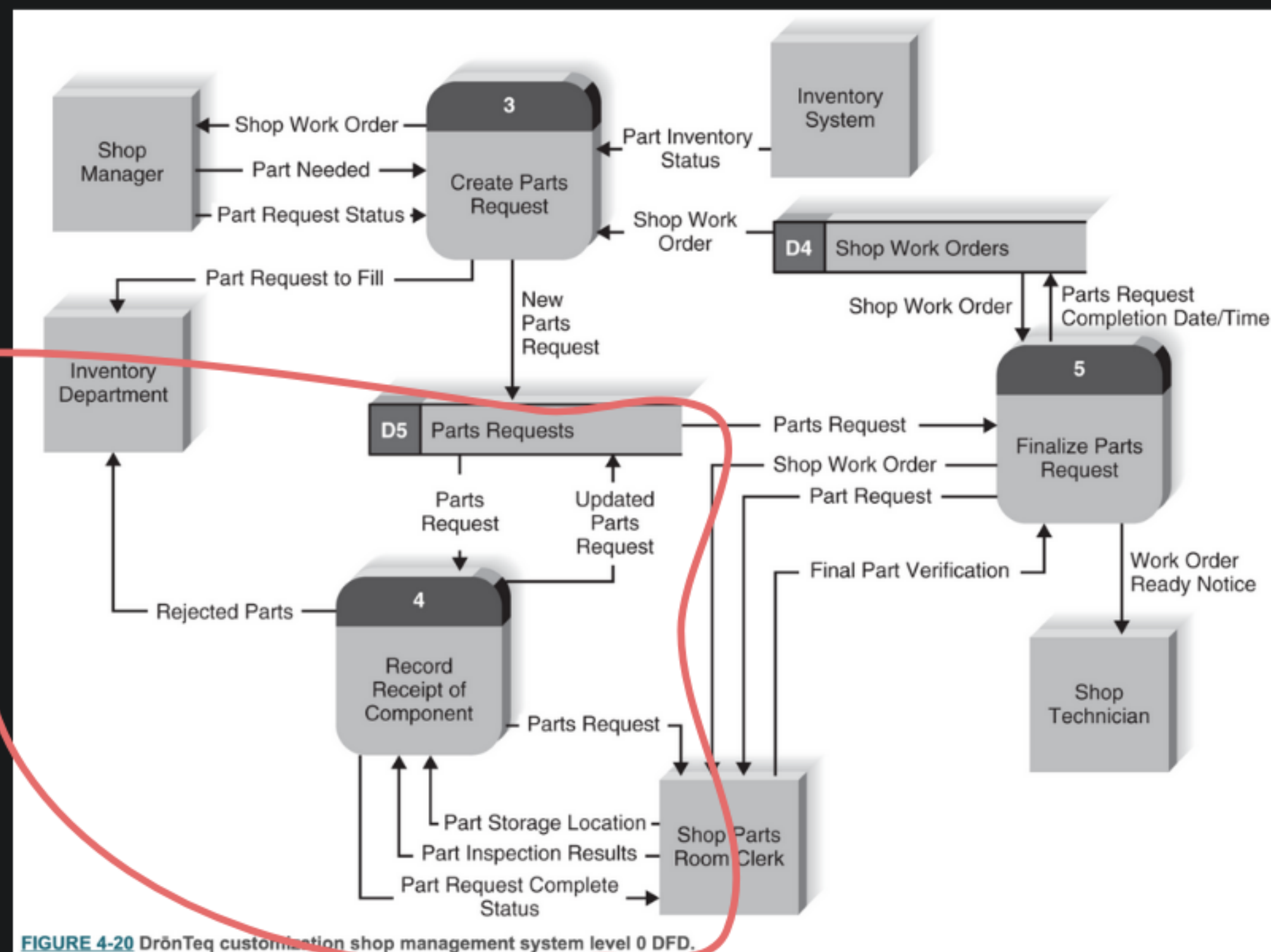


FIGURE 4-20 DrönTeq customization shop management system level 0 DFD.

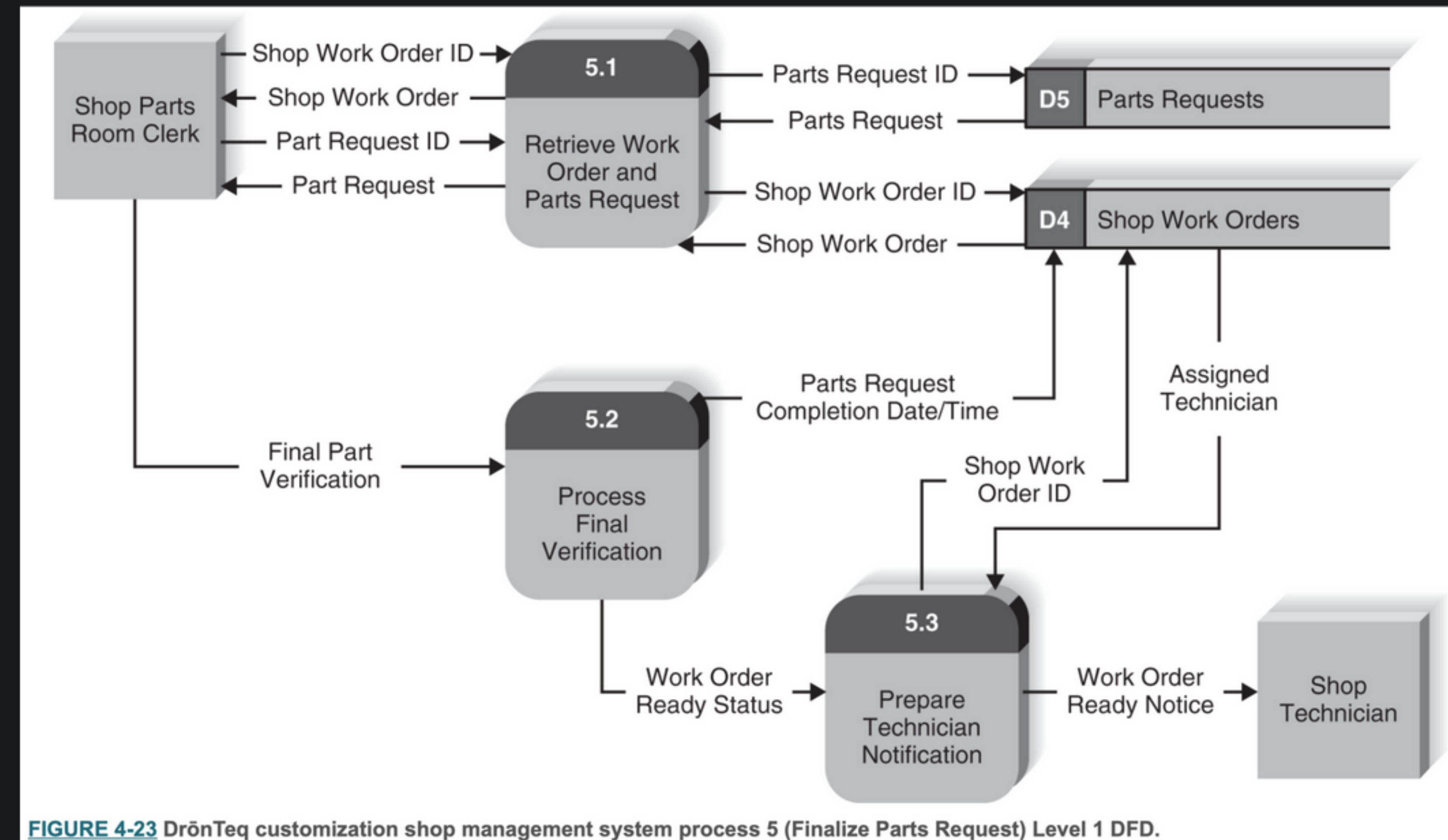
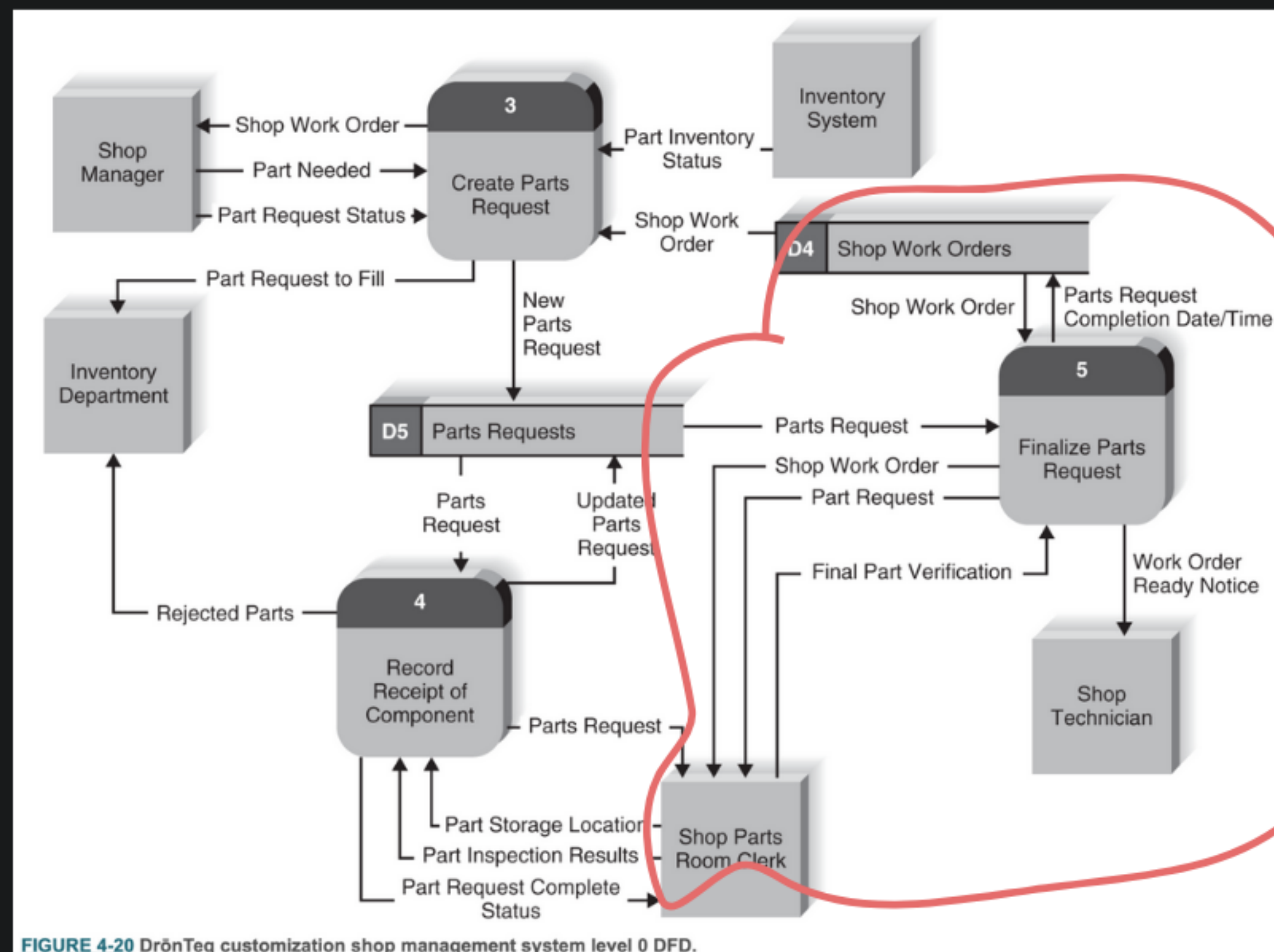
Creating Level 1 Data Flow Diagrams (and Below)



Creating Level 1 Data Flow Diagrams (and Below)



Creating Level 1 Data Flow Diagrams (and Below)



A few notes

- **3-9 processes** on every DFD
- Most systems go to only **level 2** at most
- Different processes can be decomposed to **different levels** of DFDs. Not all parts of the system need to be decomposed to the same level.
- More detailed DFDs are especially useful for a to-be model or if the system will be built by an external contractor.

Validating the Data Flow Diagrams

Syntax	
Within DFD	
Process	• Every process has a unique name that is an action-oriented verb phrase, a number, and a description. • Every process has at least one input data flow. • Every process has at least one output data flow. • Output data flows usually have different names than input data flows because the process changes the input into a different output in some way. • There are between three and seven processes per DFD.
Data Flow	• Every data flow has a unique name that is a noun, and a description. • Every data flow connects to at least one process. • Data flows only in one direction (no two-headed arrows). • A minimum number of data flow lines cross.
Data Store	• Every data store has a unique name that is a noun, and a description. • Every data store has at least one input data flow (which means to add new data or change existing data in the data store) on some page of the process model. • Every data store has at least one output data flow (which means to read data from the data store) on some page of the process model.
External Entity	• Every external entity has a unique name that is a noun, and a description. • Every external entity has at least one input or output data flow.
Across DFDs	
Context diagram	• Every set of DFDs must have one context diagram.
Viewpoint	• There is a consistent viewpoint for the entire set of DFDs.
Decomposition	• Every process is wholly and completely described by the processes on its children DFDs.
Balance	• Every data flow, data store, and external entity on a higher-level DFD is shown on the lower-level DFD that decomposes it.
Semantics	
Appropriate Representation	• User validation
	• Role-play processes
Consistent Decomposition	• Examine lowest-level DFDs
Consistent Terminology	• Examine names carefully

FIGURE 4-24 Data flow diagram quality checklist.

A few notes

- Two types of errors:
 - **Syntax errors:** Within DFD and Across DFDs
 - **Semantics errors:** whether the DFDs accurately describe the business process being modeled.
- Syntax errors are easier to find and fix. Why?
 - Although finding syntax errors across DFDs is more challenging.
 - **Most common error:** Violating the **law of conservation of data**

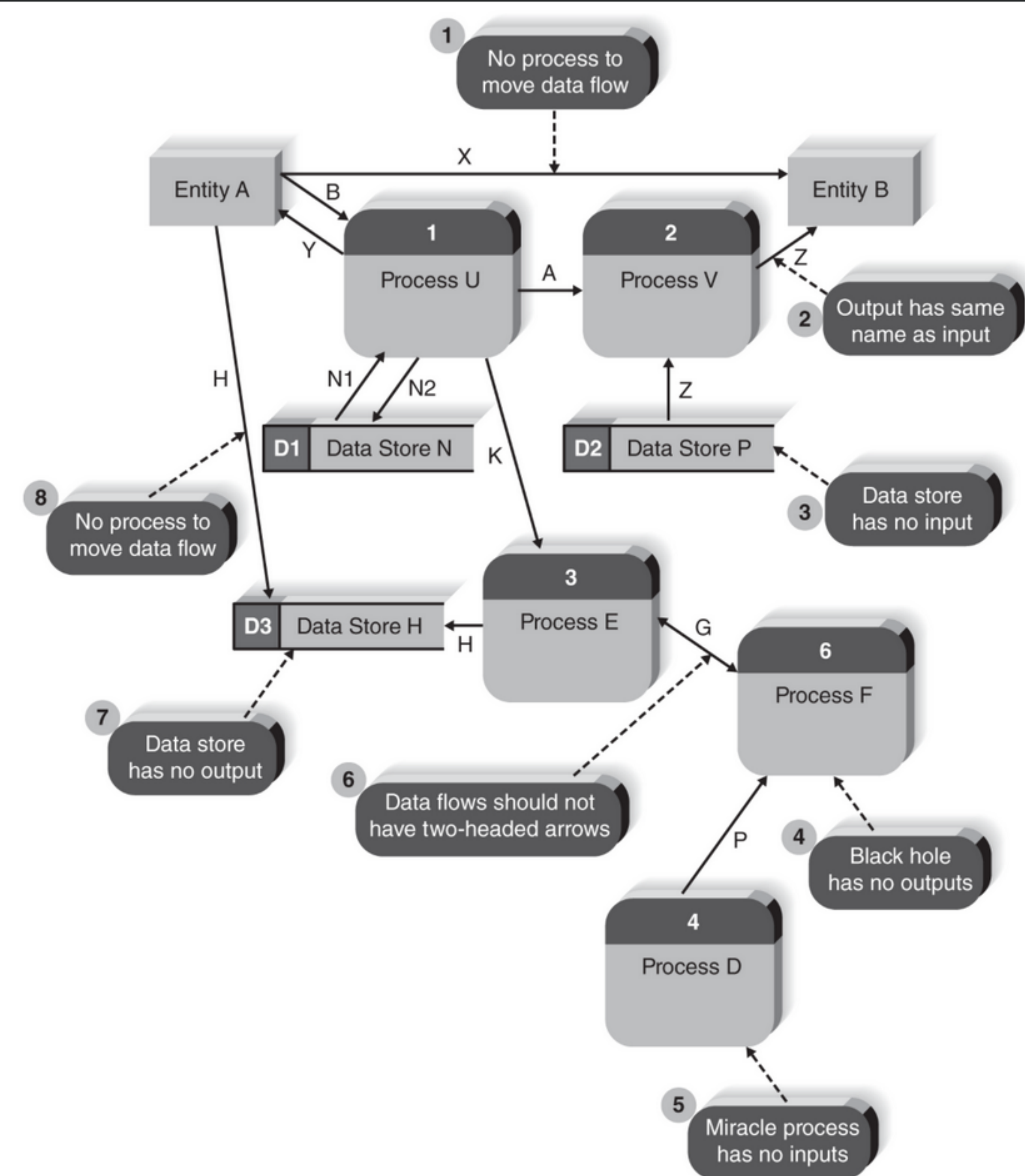


FIGURE 4-25 Some common errors.